

SPPM – Fixes for Black Spots on Glass (Checklist + Code Patches)

This document summarizes the recommended fixes to eliminate remaining black specks on specular objects (especially glass) in the hybrid SPPM (Lambertian visible points) + path tracing fallback renderer.

Symptoms

- Isolated black pixels ('pepper') concentrated on the glass sphere.
- Black pixels reduce after addressing stale pixels / one-sided emission, but do not disappear fully.
- Effect is mostly visible on specular/transmissive surfaces and near grazing angles.

Root causes (most common)

- Non-finite radiance values (NaN/Infinity) propagating through the fallback path tracer or delta chains.
- Rare samples returning zero/negative due to numerical edge cases; these clamp to black in the preview pipeline.
- Pixels not written in a given iteration (stale pixels) — mostly fixed earlier, but can still happen on early-exit paths.
- Delta material ' $1/|\cos|$ ' trick producing Inf when $\cos=0$ at grazing angles.

Fix Set A – Detect and neutralize invalid samples

Add explicit finite checks and counters so you can confirm whether black specks are caused by NaN/Inf, and prevent them from reaching the image buffer.

```
// Vec3 helpers (place in a shared utilities file)
public static class Vec3Sanity
{
    public static bool IsFinite(this Vec3 v)
        => float.IsFinite(v.X) && float.IsFinite(v.Y) && float.IsFinite(v.Z);

    public static Vec3 ClampNonNegative(this Vec3 v)
        => new Vec3(MathF.Max(0f, v.X), MathF.Max(0f, v.Y), MathF.Max(0f, v.Z));
}

// In fallback sampling site (EyePass):
Vec3 Lfb = PathTracer.EvaluateRay(primaryRay, scene, fbSampler);
if (!Lfb.IsFinite())
{
    stats.FallbackBadSamples++; // add to stats
    Lfb = Vec3.Zero;
}
else
{
    Lfb = Lfb.ClampNonNegative();
}

fallbackSum[pixelIndex] += Lfb;
fallbackCount[pixelIndex] += 1;

// In WriteBeauty (after computing final L):
```

```

if (!L.IsFinite())
{
    stats.BadBeautyPixels++;
    L = Vec3.Zero;
}
else
{
    L = L.ClampNonNegative();
}
beauty.SetPixel(x, y, L);

```

Operational guidance

- If FallbackBadSamples > 0, black specks are almost certainly due to NaN/Inf. Fix Set B becomes mandatory.
- If BadBeautyPixels > 0, sanitize the final value prior to writing the buffer to prevent UI artifacts.
- Keep these counters visible in the UI stats while iterating.

Fix Set B – Clamp delta-material cos denominators

The delta BSDF throughput trick (return $f = 1/|\cos|$ with pdf=1) is analytically correct, but numerically fragile when cos approaches 0. Clamp cos to avoid Inf/NaN.

```

// Mirror.Sample and Dielectric.Sample (delta BSDFs)
float cos = MathF.Abs(Vec3.Dot(wi, hit.Normal));
cos = MathF.Max(cos, 1e-4f); // prevents Inf at grazing angles
pdf = 1f;
f = Vec3.One / cos;          // or albedo / cos for tinted/mirror variants

```

Notes

- This clamp has negligible visual impact because it only affects extremely grazing events that are already numerically unstable.
- Apply the same clamp wherever you compute $1/\cos$ in delta sampling paths.

Fix Set C – Guarantee fallback writes for every non-VP pixel

Even with hybrid rendering, black specks can appear if some pixel paths ‘continue;’ without writing fallback (e.g., miss, delta sampling failure, or early break). Ensure fallback is produced for all non-Lambertian VP cases, including misses.

```

// In EyePass: if you do not produce a Lambertian visible point,
// always compute fallback once per pixel/iteration.

if (!gotNonDeltaHit)
{
    // fallback instead of continue-without-writing
    Vec3 Lfb = PathTracer.EvaluateRay(primaryRay, scene, fbSampler);
    if (!Lfb.IsFinite()) Lfb = Vec3.Zero;
    fallbackSum[pixelIndex] += Lfb;
    fallbackCount[pixelIndex] += 1;

    stats.VisiblePointsMissed++;
    continue;
}

// If first non-delta is non-Lambertian:
{
    Vec3 Lfb = PathTracer.EvaluateRay(primaryRay, scene, fbSampler);

```

```

    if (!Lfb.IsFinite()) Lfb = Vec3.Zero;
    fallbackSum[pixelIndex] += Lfb;
    fallbackCount[pixelIndex] += 1;

    stats.VisiblePointsSkippedNonLambertian++;
    continue;
}

```

Fix Set D – Keep fallback RNG independent of delta-chain consumption

Ensure fallback uses a derived seed so that varying random-number consumption during delta traversal does not change the fallback's sampling sequence (reduces flicker and rare black outliers).

```

// Derive a stable fallback seed from the pixel seed
ulong fbSeed = SeedHash.Hash64(seed, 0xF00DF00DUL);
var fbRng = new Pcg32(fbSeed);
var fbSampler = new Sampler(fbRng);

```

```
Vec3 Lfb = PathTracer.EvaluateRay(primaryRay, scene, fbSampler);
```

Fix Set E – Defensive guards in throughput update (optional but strong)

Add guards to stop NaN propagation early in both path tracer and photon tracer loops.

```

// After computing absCos and pdf
if (pdf <= 0f || absCos <= 0f) break;

throughput = Vec3.Hadamard(throughput, f) * (absCos / pdf);
if (!throughput.IsFinite()) break;

```

Verification checklist

- After applying Fix Set A, confirm FallbackBadSamples and BadBeautyPixels stay at 0 after several iterations.
- If black specks persist and counters remain 0, inspect whether some pixels are still unwritten (stale).
- Confirm beauty is written for every pixel each iteration: write fallback for all non-VP cases, then overwrite VP pixels.
- If specks correlate with grazing angles on glass, prioritize Fix Set B (cos clamp in delta materials).
- Optional: render a debug buffer that marks pixels where L is non-finite before sanitization.

Suggested minimal patch order

- A1: Add IsFinite/ClampNonNegative + counters (fast confirmation).
- B1: Clamp cos in delta materials (mirror + dielectric).
- C1: Ensure fallback is always written for non-VP pixels (including miss/early break).
- D1: Use a derived fallback seed independent of delta-chain RNG consumption.
- E1: Add throughput guards to prevent NaN propagation.

End of document